

CHECK POINT 01

Segurança no código, nas dependências, na infraestrutura e na aplicação em execução

njsscan, npm audit, Datree e StackHawk

Grupo 3

Leonardo Garroti

Aquiles Fonseca

Turma 2TDCPF - DevSecOps - Módulo III

Pesquisa com laboratório SAST e SCA

16 de setembro de 2026

Sumário

1. Introdução e objetivo (3)
2. SAST, SCA, IaC e DAST (4)
3. Pipeline, shift-left e SBOM (5)
4. njsscan: identificação e situação do projeto (6)
5. njsscan: funcionamento e cobertura (7)
6. njsscan: comandos, regras e integração (8)
7. npm audit: identificação e situação do projeto (9)
8. npm audit: funcionamento e leitura do resultado (10)
9. npm audit: comandos, integração e limites (11)
10. Datree: identificação e situação atual (12)
11. Datree: funcionamento e exemplos de IaC (13)
12. Datree: uso e integração planejados (14)
13. StackHawk: identificação e situação atual (15)
14. StackHawk: como o DAST funciona (16)
15. StackHawk: configuração e integração (17)
16. Laboratório e análise dos três achados (18)
17. Gate, resultados e comparação (19)
18. Avaliação crítica e conclusão (20)
- Referências (21 a 23) - Anexos (24 a 32)

1. Introdução e objetivo

Uma aplicação pode ter falhas no código, nas bibliotecas usadas, na configuração da infraestrutura e no comportamento quando está funcionando. Por isso, um único teste de segurança não cobre tudo. O objetivo deste trabalho é entender quatro formas de procurar esses problemas e usar duas delas em um laboratório pequeno, que a turma consiga acompanhar.

O grupo 3 é formado por Leonardo Garroti e Aquiles Fonseca. A tabela dos slides atribui ao grupo njsscan, npm audit, Datree e StackHawk. O arquivo de apoio descreve três grupos, mas a tabela ajustada tem cinco. Para escolher as ferramentas, este trabalho segue a coluna do grupo 3 da tabela ajustada, sem fazer substituições.

Como a pesquisa foi feita

Foram consultados repositórios, documentação dos projetos, referências da OWASP, definições CWE da MITRE e avisos da GitHub Advisory Database. Os dados de atividade foram consultados em 16 de setembro de 2026. Eles são uma fotografia da data da pesquisa e podem mudar.

O laboratório usa uma aplicação própria em Node.js. Uma versão contém três falhas de propósito e usa uma versão vulnerável do lodash. A outra aplica correções no código e atualiza a biblioteca. Foram executados njsscan e npm audit, que representam SAST e SCA. Os resultados e tempos ficaram registrados em arquivos JSON e SARIF.

No Windows, foi necessário adaptar a chamada do motor do njsscan. Essa mudança está descrita no material. Docker e GitHub Actions usam a execução normal em Linux. Datree e StackHawk foram pesquisados, mas não executados. Não foram inventadas medidas para essas duas ferramentas.

O que esperamos aprender

Ao comparar as versões, fica mais fácil entender por que o teste bloqueou uma delas, o que cada correção mudou e por que um resultado sem alertas ainda tem limites. A pesquisa também verifica se as ferramentas escolhidas continuam adequadas para uso atual, em vez de considerar apenas o nome ou a quantidade de estrelas do projeto.

Fonte da atividade: os dois PDFs fornecidos pelo professor. Apoio conceitual: NIST (2022) e OWASP (2025).

2. SAST, SCA, IaC e DAST

SAST significa teste estático de segurança da aplicação. Ele lê o código que a equipe escreveu e procura situações de risco. No nosso alvo, o njsscan encontra entrada do usuário dentro de uma resposta HTML, de um comando e de uma chamada a eval. A aplicação não precisa estar funcionando para essa análise.

SCA significa análise dos componentes usados pela aplicação. O foco está nas dependências de terceiros e em suas versões. O npm audit consulta avisos conhecidos para os pacotes do projeto. Uma aplicação pode ter um código simples e, mesmo assim, trazer uma biblioteca com uma falha publicada.

A diferença entre SAST e SCA aparece nas correções. Para resolver os três alertas SAST, mudamos o app.js. Para resolver o alerta SCA, atualizamos o lodash e o lockfile. Atualizar a biblioteca não retira o eval escrito pela equipe. Retirar o eval também não atualiza a biblioteca.

IaC é infraestrutura como código. Em vez de configurar tudo por telas, a equipe descreve recursos em arquivos. O Datree verifica configurações do Kubernetes. Ele pode apontar uma configuração que dá permissões demais a um container, antes de ela chegar ao ambiente real.

DAST significa teste dinâmico de segurança da aplicação. A ferramenta envia requisições para a aplicação funcionando e observa suas respostas. O StackHawk faz esse tipo de análise com HawkScan. O teste precisa de um alvo acessível, de um ambiente adequado e, em muitos casos, de uma configuração de autenticação.

Categoria	Objeto principal	Aplicação rodando?
SAST	Código da equipe	Não
SCA	Dependências e versões	Não
IaC	Configuração da infraestrutura	Não
DAST	Respostas da aplicação	Sim

As quatro categorias se completam. Algumas ferramentas têm mais de uma função, mas isso não muda o objeto que cada teste está analisando. Neste trabalho, a classificação segue o uso descrito acima.

Fonte: OWASP (2025), documentação das quatro ferramentas e interpretação do alvo próprio.

3. Pipeline, shift-left e SBOM

Um pipeline é uma sequência de tarefas automáticas que a equipe executa para preparar e entregar uma aplicação. Ele pode baixar o código, preparar dependências, fazer testes e liberar uma versão. Colocar segurança nessa sequência ajuda a perceber problemas antes de levar a aplicação para o usuário.

No estágio Code, SAST e IaC podem verificar o que acabou de ser escrito. Em Build, SCA verifica as dependências usadas na versão. Em Test e Release, DAST testa a aplicação funcionando. Antes de Deploy, a equipe pode conferir novamente os arquivos de infraestrutura e os valores que serão usados naquele ambiente.

Shift-left significa antecipar parte dos testes. A ideia é encontrar uma falha perto do momento em que ela foi criada. No laboratório, descobrir o eval ao analisar o código permite corrigi-lo antes da entrega. Essa antecipação ajuda, mas não observa todo o comportamento da aplicação funcionando.

DAST continua útil porque a aplicação junta código, bibliotecas, autenticação, servidor e configuração. Um erro de sessão ou uma resposta insegura pode depender dessa combinação. Por isso, nosso SAST e nosso SCA não devem ser apresentados como substitutos de um teste dinâmico.

A lista de componentes

SBOM é a sigla para Software Bill of Materials. É uma lista dos componentes usados pelo software, parecida com uma lista de ingredientes. Ela ajuda a descobrir quais versões estão presentes quando aparece um novo aviso de segurança. A lista também pode incluir dependências que chegaram por meio de outros pacotes.

Neste trabalho, `npm sbom` gerou uma SBOM no formato CycloneDX para cada alvo. Os arquivos ficam junto dos relatórios. Eles registram o inventário do lockfile. A SBOM sozinha não confirma exploração de uma falha, não corrige pacotes e não substitui a consulta a avisos que a ferramenta SCA faz.

```
npm sbom --sbom-format=cyclonedx --package-lock-only
```

Fonte: NIST (2022), OWASP (2025) e NPM, `npm-sbom` (s.d.). As SBOMs do alvo próprio acompanham a entrega.

4. njsscan: identificação e situação do projeto

O njsscan é uma ferramenta SAST voltada para aplicações Node.js. O mantenedor identificado no pacote é Ajin Abraham. A ferramenta é escrita em Python, usa libsast e chama o motor Semgrep para parte da análise. O repositório também contém muitos exemplos JavaScript, o que explica por que a API do GitHub indica JavaScript como a linguagem principal do conjunto de arquivos.

A licença identificada é LGPL-3.0. O repositório foi criado em 15 de abril de 2020. Esse dado indica o início do repositório consultado e não deve ser confundido com o início de todas as ferramentas que ele utiliza. A versão usada no laboratório foi 1.0.0, publicada no PyPI em 11 de agosto de 2026.

Atividade observada em 16/09/2026

O repositório tinha 454 estrelas e 109 forks. Esses números mostram interesse da comunidade, mas não medem quantas pessoas mantêm o projeto, a qualidade das regras ou a rapidez das correções. O relatório de atividade incluído na entrega permite conferir os dados usados.

O último commit da linha principal era de 11 de agosto de 2026. Antes dele, os commits exibidos eram de novembro de 2024. A amostra indica uma atualização recente após um intervalo longo, em vez de uma frequência constante de alterações. É um ponto que merece atenção ao escolher a ferramenta para um projeto duradouro.

A versão 1.0.0 exige Python 3.10 ou mais recente e fixa Semgrep 1.172.0. Fixar a versão reduz diferenças entre execuções, mas exige acompanhar atualizações e possíveis falhas nas próprias ferramentas do pipeline.

Por que ela entrou no laboratório

O njsscan já estava indicado para o grupo e seu foco combina com o alvo Node.js. Foi possível começar com poucos arquivos, guardar um relatório e relacionar o alerta com a linha do código. Esse tamanho ajuda quem está aprendendo. Ele também deixa claro que um teste bem sucedido num alvo pequeno não comprova cobertura de uma aplicação inteira.

Fontes: ABRAHAM, njsscan e setup.py (2026), PyPI (2026) e consulta registrada à API do GitHub.

5. njsscan: funcionamento e cobertura

O njsscan combina duas formas de procurar problemas. Uma parte usa padrões de texto, com libsast. Outra usa regras que reconhecem a estrutura do código, por meio do Semgrep. Isso permite procurar operações perigosas sem depender apenas de uma palavra isolada no arquivo.

Uma árvore de sintaxe, também chamada AST, representa a estrutura do programa. Uma regra pode procurar uma chamada de função com determinada entrada. Algumas regras usam análise de fluxo para relacionar um dado recebido com o local em que ele é usado. Essa ajuda não significa que todo alerta seja resultado de uma análise completa de todas as rotas do sistema.

O que nosso teste comprovou

No alvo próprio, apareceram três regras: `express_xss`, `generic_os_command_exec` e `eval_nodejs`. Elas apontaram três ocorrências com nível ERROR. O JSON identifica a regra, o trecho, a linha e o CWE. O SARIF permite guardar esses resultados em um formato de troca usado por plataformas de análise de código.

A entrada do usuário chega a HTML, `exec` e `eval`. A revisão do código confirmou a condição apontada em cada caso. As três ocorrências são verdadeiros positivos. Elas foram retiradas por mudanças no programa, sem comentários para ignorar as regras.

Limites encontrados

O resultado depende das regras disponíveis e da forma do código. Uma lógica de negócio errada, uma rota que libera o pedido de outro usuário ou um problema de configuração em execução podem exigir outras verificações. Não testamos aqui todas as linguagens ou classes de falha anunciadas pelo projeto.

Também houve um problema prático no Windows. `libsast 3.1.8` pula a chamada semântica nessa plataforma. Um JSON vazio, sozinho, teria levado a uma conclusão errada. A ponte local passou a chamar o Semgrep oficial com os mesmos argumentos. As regras e os exportadores do njsscan foram mantidos. O material registra a adaptação e recomenda o container Linux para a turma.

Esse caso mostrou por que devemos conferir se o scanner analisou o alvo, além de olhar apenas seu código de saída.

Fontes: código dos projetos njsscan e libsast, ABRAHAM (2026) e relatórios locais do grupo.

6. njsscan: comandos, regras e integração

No Linux, a instalação usada pelo projeto pode ser feita por pip. O laboratório fixa as versões em requirements-scanners.txt. O container prepara Python e os scanners antes da aula. A documentação também oferece uma imagem pronta, mas nosso Dockerfile reúne as duas ferramentas do laboratório numa única imagem.

```
pip install njsscan==1.0.0
njsscan targets/inseguro --json -o reports/njsscan.json
njsscan targets/inseguro --sarif -o reports/njsscan.sarif
```

JSON ajuda a ler e contar os alertas. SARIF ajuda a trocar resultados com plataformas. O projeto também tem saídas HTML e formatos voltados a SonarQube, DefectDojo e GitLab. Não testamos todas essas integrações no laboratório.

O arquivo .njsscan configura extensões, caminhos ignorados, regras ignoradas e filtros de severidade. A opção --config permite escolher outro arquivo de configuração. Uma supressão por comentário usa njsscan-ignore seguido do identificador da regra. Isso só deve acontecer depois de uma revisão que explique por que o alerta não se aplica.

Para criar uma regra nova, é preciso trabalhar com os arquivos de regras do motor e conferir o comportamento em exemplos com e sem a falha. Não há, neste laboratório, uma regra própria do grupo. Também não foi usado um baseline automático. Um baseline seria uma lista de alertas conhecidos para acompanhar mudanças, com justificativa e prazo de revisão.

Uso no trabalho da equipe

No CI, nosso script gera JSON e SARIF e aplica o gate ao JSON. GitHub Code Scanning aceita SARIF de ferramentas externas, quando disponível para o repositório. DefectDojo também tem caminhos de importação descritos pelo projeto. O fluxo entregue guarda os relatórios como artefatos, sem depender de um painel extra.

No editor, o relatório e as linhas apontadas ajudam a revisão. Um hook local antes do commit pode chamar o scanner, mas a equipe precisa ter o ambiente instalado. Esse teste local facilita corrigir cedo. O CI continua necessário porque nem todo colaborador usa o hook.

Ponto forte: revisão simples do alvo Node.js. Limite: cobertura das regras e cuidado com o ambiente. Alternativas incluem Semgrep direto e outros analisadores de JavaScript. Não houve troca porque a ferramenta foi atribuída ao grupo.

Fontes: ABRAHAM, README e interface de linha de comando (2026), GITHUB, SARIF support (s.d.).

7. npm audit: identificação e situação do projeto

npm audit é um comando do npm, o gerenciador de pacotes usado por muitos projetos Node.js. O mantenedor do conjunto npm CLI é npm, Inc. e sua comunidade de colaboradores. O código da CLI é JavaScript. A licença principal da aplicação npm é Artistic License 2.0, enquanto seus próprios pacotes de dependência mantêm suas respectivas licenças.

O comando audit apareceu na geração npm 6, em 2018. O repositório atual npm/cli também foi criado em 2018, mas o npm já existia antes. Por isso, o ano do comando não representa o nascimento de todo o gerenciador de pacotes.

Atividade observada

Em 16 de setembro de 2026, o repositório npm/cli tinha 10.118 estrelas e 4.730 forks. Os commits mais recentes na linha principal estavam em 3 de setembro, 31 de agosto e 26 de agosto de 2026. Essa amostra mostra alterações próximas entre si. Ela não mede a quantidade total de colaboradores ativos.

O pacote npm tinha a versão estável mais recente 12.0.2, publicada em 29 de julho de 2026, segundo o registro consultado. Usamos npm 11.12.0, publicado em 18 de março de 2026, para manter a mesma versão nos comandos da preparação, no Dockerfile e no CI. A versão escolhida é compatível com o Node 24 do ambiente.

Por que usar no laboratório

A vantagem para quem está aprendendo é que o comando já pertence ao gerenciador de pacotes. Ele permite enxergar a relação entre package.json, package-lock.json e avisos conhecidos. Não é necessário instalar uma plataforma extra para obter o relatório básico.

O lockfile registra as versões resolvidas, incluindo dependências de outras dependências. Manter esse arquivo junto do código ajuda a comparar as duas versões do alvo. Neste trabalho, a mudança do lodash de 4.17.15 para 4.18.1 ficou registrada nos arquivos de pacote e no lockfile.

Estrelas e frequência de commits não garantem que a base conheça toda falha possível. O audit aponta avisos disponíveis para versões identificadas, e não uma revisão completa do código da equipe.

Fontes: NPM, changelog v6 (2018), LICENSE e documentação de npm audit (s.d.), registro npm e atividade registrada do GitHub.

8. npm audit: funcionamento e leitura do resultado

O npm audit consulta o registro configurado para procurar avisos de segurança das dependências. No laboratório, o registro usado é o público do npm. A consulta envolve nomes e versões dos pacotes. Ela não exige que a aplicação esteja funcionando e não executa ataques contra a aplicação.

O scanner compara as versões do projeto com as faixas afetadas nos avisos. Um pacote pode trazer outra dependência vulnerável. Isso explica por que também precisamos olhar dependências indiretas e não apenas a lista escrita no package.json.

O resultado da versão insegura

O relatório apontou o pacote lodash como HIGH. A contagem final foi de um pacote vulnerável. Dentro da lista via, havia seis avisos diretos na data do teste. Portanto, um pacote HIGH não significa que o relatório tenha apenas um aviso ou um único CVE.

Um exemplo é CVE-2020-8203, sobre Prototype Pollution, com CWE-1321 e CVSS 7,4 no aviso consultado. Outro é CVE-2026-4800, com CWE-94 e CVSS 8,1. Esse segundo aviso mostra por que simplesmente atualizar para uma versão que parecia segura anos atrás pode não resolver os avisos disponíveis hoje.

O relatório recomendou uma atualização para lodash 4.18.1. Essa versão foi fixada no alvo corrigido. Na nova execução, o npm audit não apontou pacotes vulneráveis. O resultado depende da data e da base de avisos, que pode receber atualizações.

Presença e exploração são questões diferentes

A versão vulnerável estava presente no lockfile. Esse é um verdadeiro positivo de inventário. Nosso código usa a função chunk, e o laboratório não demonstrou que as funções citadas nos avisos sejam exploráveis nessa rota. Não chamamos isso de exploração confirmada.

O npm audit não encontra o eval que a equipe escreveu. Também não serve, sozinho, como análise de licença, lista completa de pacotes maliciosos ou prova de que a aplicação está segura. Para esses objetivos, a equipe precisa de outras fontes e verificações.

Fontes: NPM, npm-audit (s.d.), GitHub Advisory Database (2020; 2026) e relatórios reais dos dois alvos.

9. npm audit: comandos, integração e limites

O npm costuma vir com Node.js e também pode ser atualizado separadamente. No nosso Dockerfile e no CI, a versão do npm é fixada em 11.12.0. O laboratório consulta o lockfile, por isso não precisa executar o servidor nem instalar e rodar todos os pacotes do alvo para obter o audit.

```
npm install --package-lock-only --ignore-scripts --no-audit
npm audit --json --audit-level=high --ignore-scripts
npm audit fix --dry-run --json
```

--json produz o relatório estruturado. --audit-level=high define o nível mínimo que faz o comando terminar com erro. Essa opção não esconde os avisos de níveis menores. --ignore-scripts evita scripts de instalação quando há uma operação de pacote. O audit em si consulta dados das dependências.

audit fix tenta ajustar versões, mas precisa de revisão. Algumas mudanças podem alterar o funcionamento da aplicação. --dry-run permite observar uma proposta sem aplicá-la. Não usamos --force para aceitar mudanças sem conferir. A correção do laboratório ficou explícita nos arquivos e foi analisada novamente.

Regras e exceções

O npm audit não oferece regras próprias para encontrar padrões de código. Sua base é o conjunto de avisos do registro. Também não usa comentários no app.js para suprimir um CVE. Uma exceção de risco exigiria uma decisão da equipe, com aviso identificado, motivo, responsável e data de revisão. Não houve exceção no nosso gate.

No CI, o comando combina com a instalação de dependências e com um script que verifica HIGH e CRITICAL. No editor ou no pre-commit, a equipe pode chamar o mesmo comando, considerando o tempo e a necessidade de internet. A CLI entrega JSON e texto. Não tratamos sua saída como SARIF nativo. Um uso em Code Scanning precisa de conversão ou integração adicional.

Ponto forte: pouca preparação extra em projetos npm. Limites: base de avisos, rede e ausência de análise da rota que usa cada função. Alternativas de SCA incluem OSV-Scanner e soluções que aceitam SBOM. A escolha do laboratório segue a ferramenta atribuída e o alvo Node.js.

Fontes: NPM, npm-audit, npm-sbom e documentação de auditoria (s.d.).

10. Datree: identificação e situação atual

Datree é uma ferramenta de verificação de políticas para configurações Kubernetes. O projeto foi mantido por Datree e colaboradores, é escrito principalmente em Go e usa a licença Apache-2.0. O repositório consultado foi criado em 14 de abril de 2021.

A versão mais recente publicada no repositório é 1.9.19, de 23 de julho de 2023. Em 16 de setembro de 2026, o projeto tinha 6.331 estrelas e 356 forks. Essa quantidade de interesse acumulado não representa manutenção atual.

Um problema para a escolha atual

O README informa que a empresa que mantinha o projeto encerrou suas atividades em julho de 2023. A linha principal mostra um aviso de descontinuação em agosto de 2023. O GitHub indica que o repositório foi arquivado em 6 de junho de 2024 e está somente para leitura.

O campo de última atualização do repositório e a data do último commit da linha principal podem ser diferentes. Por isso, a pesquisa não usa apenas um campo de atividade para dizer que o projeto recebe correções. O aviso explícito de descontinuação é a informação mais relevante para esta avaliação.

Datree continua sendo um exemplo útil para entender a verificação de configuração por políticas. Contudo, a equipe precisa considerar a ausência de novas regras e de correções no próprio verificador. As versões recentes do Kubernetes podem mudar esquemas e recursos que uma ferramenta antiga não conhece.

Decisão no trabalho

Datree aparece na coluna do grupo 3, então foi mantido na pesquisa. A ferramenta não foi substituída sem autorização do professor. Também não foi escolhida como uma das duas execuções obrigatórias, porque o laboratório usa njsscan e npm audit.

Não há taxa de falso positivo ou tempo medido de Datree neste trabalho. Os manifests de exemplo ajudam a discutir configurações, mas não são apresentados como um relatório de execução dessa ferramenta.

Para uma implantação atual, seria preciso escolher uma ferramenta mantida e verificar se ela atende às políticas da equipe. OPA/Conftest e verificadores atuais de Kubernetes são alternativas para essa discussão, sujeitas às regras de substituição da atividade.

Fontes: DATREE, repositório, README e release 1.9.19 (2023; 2024), consulta à API do GitHub.

11. Datree: funcionamento e exemplos de IaC

Datree verifica arquivos Kubernetes antes de aplicar a configuração no ambiente. Seu trabalho principal é comparar os recursos descritos com políticas. A documentação separa validação de YAML, validação de esquema Kubernetes e verificação das regras da política.

Uma validação de YAML verifica se o arquivo pode ser lido no formato esperado. Uma validação de esquema verifica se a estrutura combina com o tipo e a versão do recurso. Uma política pode verificar se um campo permite mais acesso do que a equipe deseja, mesmo que o arquivo tenha uma estrutura válida.

Nosso exemplo de configuração

O manifesto inseguro do material usa `privileged: true`, usuário 0, `hostNetwork: true` e uma imagem com a tag `latest`. São configurações escritas de propósito para a discussão. Elas podem aumentar o acesso do container ou tornar a versão da imagem menos previsível.

O manifesto melhorado retira o modo privilegiado e o compartilhamento de rede do host. Também usa execução sem root, bloqueia aumento de privilégios, retira capabilities e define limites de recursos. Essas escolhas ilustram a intenção de dar somente as permissões necessárias.

Esse exemplo não foi aplicado a um cluster. Também não houve execução de Datree sobre ele. A palavra corrigido no nome indica as mudanças discutidas, sem afirmar que o manifesto passou por todas as regras da ferramenta ou por um teste real de funcionamento da imagem.

O que essa análise não vê

Ler o manifesto não confirma que a aplicação trata corretamente a entrada do usuário. Também não confirma a presença de CVEs nas dependências de uma aplicação. São objetivos de outros testes. Uma configuração segura no arquivo pode ainda depender de valores, permissões e condições do ambiente em que será aplicada.

A documentação histórica cita suporte a políticas próprias, JSON Schema, Rego, Helm e Kustomize. Isso ajuda a entender a abordagem. A compatibilidade de cada recurso deve ser conferida com a versão fixada, especialmente porque o projeto foi descontinuado.

No modo offline, a validação de esquema precisa receber esquemas locais quando desejada. O simples uso sem internet não traz automaticamente um conhecimento atualizado sobre novas versões do Kubernetes.

Fontes: DATREE, offline mode, CLI arguments e README (s.d.; 2023). Os manifests são exemplos próprios.

12. Datree: uso e integração planejados

O repositório de releases oferece arquivos binários para várias plataformas. A versão estudada é 1.9.19. Para preparar um uso didático, é necessário baixar o arquivo da plataforma correta, conferir sua origem e trabalhar com uma configuração local. Não usamos instaladores antigos que dependem de um serviço encerrado.

```
datree version
datree config set offline local
datree test iac/pod-inseguro.yaml --output json
datree test iac/pod-inseguro.yaml --output sarif
```

Esses comandos são exemplos baseados na documentação histórica. Eles não são evidência de uma execução feita pelo grupo. A configuração offline permite trabalhar com as políticas locais compiladas no binário, mas não mantém o antigo painel ou o registro central de políticas.

--policy-config permite escolher uma política em arquivo. --policy escolhe a política por nome. --schema-location permite informar os esquemas. --no-record evita o envio de registros, mas não é o mesmo que preparar um ambiente totalmente sem internet. A documentação lista saídas JSON, YAML, XML, JUnit, SARIF e texto simples.

Regras próprias e alertas antigos

Uma política própria expressa as condições que a equipe deseja exigir. Se uma regra não se aplica a um recurso, a equipe precisa explicar a exceção e limitar seu alcance. Uma lista de exceções não deve esconder mudanças de configuração. O trabalho não cria nem aplica um baseline de Datree.

No CI, a CLI pode rodar antes da aplicação de manifests. Um hook local também pode conferir o arquivo alterado. No editor, o relatório ajuda a localizar a configuração. SARIF pode servir para plataformas que aceitam esse formato, mas a integração exata precisa ser testada. Não há aqui validação de um painel de Datree, Code Scanning ou DefectDojo para essa ferramenta.

Ponto forte da abordagem: regras claras para configuração Kubernetes, sem exigir o cluster em execução. Limite atual: manutenção encerrada. Não é correto transformar automaticamente toda violação de política em uma nota CVSS ou em um alerta HIGH. Isso precisaria de uma política de severidade própria, que não foi incluída no nosso gate.

Fontes: DATREE, CLI arguments, offline mode e release 1.9.19 (s.d.; 2023).

13. StackHawk: identificação e situação atual

StackHawk oferece teste dinâmico de segurança para aplicações e APIs. O produto tem o scanner HawkScan e a plataforma StackHawk, onde os resultados podem ser acompanhados. O fornecedor é StackHawk, Inc. A empresa relata a formação de sua equipe em julho de 2019.

A documentação consultada indica HawkScan 6.4.0 como versão atual. O motor é distribuído para execução local ou em pipeline. A geração 6 usa um lançador em Rust e inclui o ambiente Java necessário ao motor. Como o código completo do produto não é um repositório público aberto, não atribuímos uma linguagem única a todas as partes da plataforma.

Origem no ZAP e licença

O fornecedor informa que HawkScan se originou do ZAP e que passou a usar seu próprio motor HSTE, a partir da geração 4.0. Essa origem não permite tratar todo o produto comercial como se tivesse a mesma licença e o mesmo modelo de manutenção do ZAP.

A plataforma é um serviço comercial. Não foi encontrada uma licença aberta única que torne todo o conjunto StackHawk um projeto open source. Repositórios públicos de exemplos ou integrações não provam que o motor completo ou o serviço tenham código aberto. A pesquisa deixa essa diferença explícita.

Não há número de estrelas, frequência de commits ou tamanho de comunidade pública do produto inteiro comparável aos outros três repositórios. Para avaliar atividade, usamos o changelog e a versão publicada pelo fornecedor, sem inventar métricas de um repositório inexistente.

Decisão no trabalho

StackHawk foi mantido porque consta na tabela do grupo 3. Sua classificação comercial entra na avaliação crítica. A atividade geral pede ferramentas abertas, então essa diferença precisa ser conversada com o professor. Não houve uma substituição silenciosa por ZAP ou por outra ferramenta atribuída a um grupo diferente.

A ferramenta não foi executada neste laboratório. Não há conta, chave, relatório DAST, tempo medido ou taxa de falso positivo produzidos pelo grupo para ela.

Fontes: STACKHAWK, HawkScan, changelog, anúncio do HSTE (2024), anúncio da empresa (2022) e notas HawkScan 6.0.0 (2026).

14. StackHawk: como o DAST funciona

HawkScan precisa se conectar a uma aplicação que esteja funcionando. Primeiro, a configuração indica o alvo e a forma de acessar suas rotas. O scanner pode usar navegação, informações de API e autenticação para alcançar partes da aplicação. Depois, ele faz requisições de teste e observa as respostas.

Uma análise passiva observa características das respostas. Uma análise ativa envia entradas para tentar revelar comportamentos inseguros. O fornecedor avisa que a análise ativa pode criar, alterar e apagar dados do alvo. Por isso, um ambiente de teste deve ser preparado para esse tipo de uso.

Código parado e aplicação funcionando

No nosso SAST, o código mostra que a entrada chega a eval. Um DAST avaliaria a aplicação pela rede e precisaria alcançar a rota. Se a autenticação ou a configuração impedir o acesso, a cobertura pode ser menor. A ferramenta pode terminar a análise sem ter visto a parte mais importante do sistema.

Essa diferença ajuda a entender a necessidade de informar rotas, sessões de teste e descrições de API, quando o alvo usa esses recursos. DAST depende tanto do scanner quanto de uma configuração de acesso que corresponda ao funcionamento real da aplicação.

O fornecedor apresenta cobertura para falhas como injeções e problemas de configuração web. Neste trabalho, essas capacidades são informações da documentação, não achados do laboratório. Não executamos HawkScan no nosso alvo e não apresentamos uma captura de uma exploração que não ocorreu.

Limites de um teste dinâmico

Um teste pela rede não faz um inventário completo das dependências instaladas. Também pode não compreender regras de negócio como aprovação de uma compra ou acesso a dados de outro usuário. Um resultado sem alertas depende das rotas e das condições que o scanner conseguiu testar.

Se for usado depois pelo grupo, o alvo deve ser a aplicação própria num ambiente local isolado, sem dados reais. A publicação do repositório não dá autorização para varrer sites públicos ou a rede da escola. Este trabalho não executou varredura ativa em terceiros.

Fontes: STACKHAWK, HawkScan, configuração, FAQ e troubleshooting (s.d.). Comparação baseada no alvo próprio.

15. StackHawk: configuração e integração

A documentação oferece HawkScan por CLI e por container. A CLI usa o comando `hawk scan`. O container oficial é `stackhawk/hawkscan`. A ferramenta também precisa de configuração ligada à plataforma e de uma chave de acesso. Nenhuma chave foi criada, copiada ou incluída neste trabalho.

```
docker pull stackhawk/hawkscan:6.4.0
hawk version
hawk scan --sarif-artifact
```

Esses comandos ilustram a preparação e a execução descritas pelo fornecedor. Não fazem parte das duas ferramentas obrigatórias do laboratório. Antes de executá-los, a dupla precisaria configurar sua conta e confirmar o alvo próprio que vai receber o teste.

Um arquivo `stackhawk.yml` informa o identificador da aplicação, o ambiente e o endereço do alvo. A configuração também pode tratar autenticação, descrição OpenAPI e exclusões de rotas. Uma exclusão deve ser justificada, pois ela pode retirar cobertura de uma parte do sistema.

Regras, exceções e formatos

A ferramenta permite configurar testes e a forma de alcançar o alvo. Isso não equivale a escrever regras SAST sobre o código. A triagem na plataforma ajuda a acompanhar achados conhecidos e decisões sobre falsos positivos. Não produzimos um baseline ou uma exceção de HawkScan neste trabalho.

A CLI tem opção para salvar um artefato SARIF. Os resultados também são acompanhados na plataforma, e existem integrações de CI e GitHub documentadas pelo fornecedor. O caminho de importação em outra plataforma, como DefectDojo, deve ser verificado pelo formato e pela versão, sem presumir que qualquer JSON seja compatível.

DAST geralmente entra depois que o ambiente de teste está pronto. Rodá-lo em todo pre-commit pode demorar e exige uma aplicação acessível. O uso no editor pode acontecer por comandos e integrações, mas não transforma o teste dinâmico em leitura estática do código.

Ponto forte proposto: organização de testes e resultados para aplicações e APIs. Limites neste trabalho: conta e chave ausentes, execução não realizada e produto comercial. ZAP e outros scanners abertos são alternativas para avaliar com o professor, considerando as ferramentas já atribuídas aos demais grupos.

Fontes: STACKHAWK, CLI, configuração, instalação e integrações (s.d.).

16. Laboratório e análise dos três achados

O alvo próprio tem dois conjuntos de arquivos: inseguro e corrigido. Os dois usam Express 5.2.1. O alvo inseguro usa lodash 4.17.15, e o corrigido usa 4.18.1. As versões exatas e as dependências resolvidas estão nos lockfiles. Nenhum sistema de terceiros recebeu teste dinâmico.

SAST e SCA não exigem o servidor funcionando. Nosso Compose executa os scanners e compartilha os relatórios, sem publicar portas. A imagem precisa ser preparada antes da aula. O roteiro LAB.md organiza a execução em 12 minutos, incluindo a leitura dos resultados e as perguntas da turma.

Os três achados revisados

`express_xss` aparece nas linhas 8 a 9 do `app.js` inseguro. A entrada nome vai para HTML sem escape. A classificação é verdadeiro positivo, com CWE-79. A correção devolve texto simples. Assim, a resposta mantém a função de mostrar o nome sem tratar a entrada como HTML.

`generic_os_command_exec` aparece nas linhas 12 a 14. O usuário escolhe o texto recebido por `exec`. A classificação é verdadeiro positivo, com CWE-78. A correção retira a execução de comando e devolve uma mensagem fixa. Não foi preciso enviar um comando perigoso para confirmar o caminho no código.

`eval_nodejs` aparece na linha 17. O usuário controla a expressão avaliada como código. A classificação é verdadeiro positivo, com CWE-95. A correção aceita dois números, valida os valores e soma somente esses números, sem `eval`.

Na amostra, foram zero falsos positivos em três alertas SAST revisados, ou 0%. Como as falhas foram colocadas de propósito num alvo pequeno, essa taxa não pode ser generalizada para projetos reais. Para `npm audit`, confirmamos a presença da versão vulnerável, sem afirmar exploração da biblioteca na rota `chunk`.

Os três alertas SAST se relacionam a A05:2025 - Injection. O grupo fez esse mapeamento para a edição atual, porque algumas regras preservam rótulos OWASP antigos. Os exemplos próprios não receberam CVE nem nota CVSS do scanner.

Fontes: relatórios do grupo, MITRE CWE-78, CWE-79 e CWE-95 (s.d.), OWASP (2025).

17. Gate, resultados e comparação

O gate é a regra que decide se os testes permitem aprovar uma versão. Nosso script lê os dois JSONs. No npm audit, HIGH e CRITICAL bloqueiam. No njsscan, ERROR é tratado como HIGH por decisão do grupo. WARNING e INFO continuam visíveis, mas não bloqueiam por essa política.

Essa tradução do nível SAST não cria uma nota CVSS. Ela apenas define uma decisão de aprovação para o laboratório. Na versão insegura, três ocorrências ERROR e um pacote HIGH fizeram o gate terminar com código 1. Na versão corrigida, os dois relatórios ficaram sem alertas e o gate terminou com código 0.

Ferramenta	Uso principal	Resultado no nosso alvo
njsscan	Código Node.js	3 ERROR, depois 0
npm audit	Dependências npm	1 pacote HIGH, depois 0
Datree	Manifests Kubernetes	Não executado
StackHawk	Aplicação em execução	Não executado

Os tempos medidos de cada execução aparecem em execucao.json. Eles dependem do tamanho do alvo, do computador e da rede. A geração de SARIF faz uma segunda análise, registrada separadamente. Não é um teste de desempenho entre as quatro ferramentas.

Integração preparada

O fluxo do GitHub Actions usa Linux, Python 3.12 e Node 24. Em push ou pull request, ele analisa o alvo corrigido. A execução manual permite escolher inseguro ou corrigido. Isso prepara a demonstração de um build vermelho e um verde, com os relatórios guardados como artefatos.

Relatório ausente, formato inválido ou erro de execução também bloqueiam. Uma falha de rede não deve ser apresentada como zero falhas de segurança. O fluxo guarda os relatórios mesmo quando o gate bloqueia, para permitir a revisão do motivo.

Os resultados locais comprovam a execução dos scanners e do gate local. Eles não comprovam publicação ou execução no GitHub. Os builds da plataforma precisam ser verificados depois da publicação. Da mesma forma, o Dockerfile e o Compose precisam de teste do zero num computador com Docker, que não estava disponível na preparação.

Fonte: scripts, relatórios e fluxo preparados pelo grupo. Referências de integração: documentação oficial das ferramentas.

18. Avaliação crítica e conclusão

O aprendizado principal foi separar o problema do código do problema da dependência. Os três erros do app.js exigiram mudanças no programa. O aviso de lodash exigiu uma atualização no conjunto de pacotes. Fazer apenas uma dessas ações teria deixado parte do problema sem correção.

O njsscan foi útil para ligar um alerta a uma linha do código num alvo Node.js pequeno. O problema encontrado no Windows mostrou uma limitação concreta: um relatório vazio pode acontecer porque uma etapa não rodou. Conferir o motor e registrar a adaptação foi necessário para obter uma conclusão correta.

O npm audit exigiu pouca preparação e apontou uma versão com avisos conhecidos. Sua contagem também exigiu cuidado. Havia um pacote HIGH reunindo vários avisos. A lista de componentes confirmou a versão, mas não mostrou que cada falha era explorável na rota usada pelo alvo.

Datree ajuda a explicar políticas de configuração Kubernetes, mas sua manutenção encerrada limita uma recomendação atual. StackHawk ajuda a explicar DAST, porém é uma plataforma comercial e não foi executado pelo grupo. Essas diferenças precisam ser tratadas com o professor porque a atividade geral fala em ferramentas abertas.

Uma sequência possível para uma equipe

Para um projeto Node.js, a equipe pode analisar o código e as dependências cedo e repetir os testes no CI. Se usar Kubernetes, também deve verificar os manifests com uma ferramenta mantida. Depois que a aplicação de teste estiver pronta, pode incluir DAST com um alvo e uma configuração de acesso bem definidos.

A recomendação é combinar as categorias de acordo com a aplicação e revisar os resultados. Nenhuma ferramenta garante sozinha a segurança do sistema. SAST, SCA, IaC e DAST observam objetos diferentes, e seus resultados dependem das regras, da base de avisos, das entradas e do ambiente.

Antes da entrega, a dupla ainda precisa realizar a revisão conjunta, testar Docker numa máquina adequada, conferir os builds da plataforma e ensaiar dentro de 30 minutos. Essas ações reais não podem ser substituídas por texto, por um histórico de commits inventado ou por prints de outra execução.

Fontes: resultados locais do grupo e fontes oficiais citadas ao longo da pesquisa. O apoio de IA e os limites de validação estão declarados no anexo.

Referências

ABRAHAM, Ajin. **njsscan**. 2026. Disponível em: <https://github.com/ajinabraham/njsscan>. Acesso em: 16 set. 2026.

ABRAHAM, Ajin. **njsscan: setup.py**. 2026. Disponível em: <https://github.com/ajinabraham/njsscan/blob/master/setup.py>. Acesso em: 16 set. 2026.

ABRAHAM, Ajin. **njsscan: interface de linha de comando**. 2026. Disponível em: https://github.com/ajinabraham/njsscan/blob/master/njsscan/__main__.py. Acesso em: 16 set. 2026.

ABRAHAM, Ajin. **libsast: chamada do motor Semgrep**. s.d.. Disponível em: https://github.com/ajinabraham/libsast/blob/master/libsast/core_sgrep/helpers.py. Acesso em: 16 set. 2026.

DATREE. **Datree: aviso de descontinuação**. 2023. Disponível em: <https://github.com/datreeio/datree>. Acesso em: 16 set. 2026.

DATREE. **Release 1.9.19**. 2023. Disponível em: <https://github.com/datreeio/datree/releases/tag/1.9.19>. Acesso em: 16 set. 2026.

DATREE. **Offline mode**. s.d.. Disponível em: <https://hub.datree.io/cli/offline-mode>. Acesso em: 16 set. 2026.

DATREE. **CLI arguments**. s.d.. Disponível em: <https://hub.datree.io/cli/cli-arguments>. Acesso em: 16 set. 2026.

GITHUB. **SARIF support for code scanning**. s.d.. Disponível em: <https://docs.github.com/en/code-security/reference/code-scanning/sarif-files/sarif-support>. Acesso em: 16 set. 2026.

GITHUB ADVISORY DATABASE. **Prototype Pollution in Iodash: CVE-2020-8203**. 2020. Disponível em: <https://github.com/advisories/GHSA-p6mc-m468-83gw>. Acesso em: 16 set. 2026.

Referências (continuação)

GITHUB ADVISORY DATABASE. **Lodash vulnerable to Code Injection: CVE-2026-4800**. 2026. Disponível em: <https://github.com/advisories/GHSA-r5fr-rjxr-66jc>. Acesso em: 16 set. 2026.

MITRE. **CWE-78: OS Command Injection**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/78.html>. Acesso em: 16 set. 2026.

MITRE. **CWE-79: Cross-site Scripting**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/79.html>. Acesso em: 16 set. 2026.

MITRE. **CWE-95: Eval Injection**. s.d.. Disponível em: <https://cwe.mitre.org/data/definitions/95.html>. Acesso em: 16 set. 2026.

NIST. **Secure Software Development Framework (SSDF), SP 800-218**. 2022. Disponível em: <https://csrc.nist.gov/pubs/sp/800/218/final>. Acesso em: 16 set. 2026.

NPM. **npm-audit**. s.d.. Disponível em: <https://docs.npmjs.com/cli/v11/commands/npm-audit/>. Acesso em: 16 set. 2026.

NPM. **npm-sbom**. s.d.. Disponível em: <https://docs.npmjs.com/cli/v11/commands/npm-sbom/>. Acesso em: 16 set. 2026.

NPM. **Changelog: npm v6**. 2018. Disponível em: <https://docs.npmjs.com/cli/v6/using-npm/changelog/>. Acesso em: 16 set. 2026.

NPM. **Licença da aplicação npm**. s.d.. Disponível em: <https://github.com/npm/cli/blob/latest/LICENSE>. Acesso em: 16 set. 2026.

NPM. **Registro do pacote npm**. 2026. Disponível em: <https://registry.npmjs.org/npm>. Acesso em: 16 set. 2026.

Referências (continuação)

OWASP. **Top 10:2025**. 2025. Disponível em: https://top10.owasp.org/2025/0x00_2025-Introduction/. Acesso em: 16 set. 2026.

PYPI. **njsscan 1.0.0**. 2026. Disponível em: <https://pypi.org/project/njsscan/1.0.0/>. Acesso em: 16 set. 2026.

STACKHAWK. **HawkScan**. s.d.. Disponível em: <https://docs.stackhawk.com/hawkscan/>. Acesso em: 16 set. 2026.

STACKHAWK. **StackHawk CLI**. s.d.. Disponível em: <https://docs.stackhawk.com/stackhawk-cli/>. Acesso em: 16 set. 2026.

STACKHAWK. **HawkScan Configuration**. s.d.. Disponível em: <https://docs.stackhawk.com/hawkscan/configuration/>. Acesso em: 16 set. 2026.

STACKHAWK. **HawkScan 6.0.0 Release Notes**. 2026. Disponível em: <https://docs.stackhawk.com/hawkscan/6-0-0/>. Acesso em: 16 set. 2026.

STACKHAWK. **Changelog**. s.d.. Disponível em: <https://docs.stackhawk.com/changelog/>. Acesso em: 16 set. 2026.

STACKHAWK. **StackHawk Announces HawkScan Test Engine**. 2024. Disponível em: <https://www.stackhawk.com/blog/stackhawk-announces-hawkscan-test-engine/>. Acesso em: 16 set. 2026.

STACKHAWK. **Announcing Series B Funding**. 2022. Disponível em: <https://www.stackhawk.com/blog/announcing-stackhawks-207-million-in-series-b-funding-to-drive-developer-first-security/>. Acesso em: 16 set. 2026.

Anexo A. Uso de IA e validação

Este material foi preparado com apoio do Codex, uma ferramenta de IA. A IA ajudou a ler os enunciados, consultar fontes oficiais, escrever a primeira versão da pesquisa e dos slides, preparar o alvo próprio e executar os scanners. A declaração completa está em USO-DE-IA.md.

Os relatórios reais foram comparados com o código e com as referências CWE. A versão corrigida foi analisada novamente. As fontes dos projetos foram usadas para conferir versões, licenças e situação de manutenção. A pesquisa preserva as diferenças encontradas no enunciado.

Limites da preparação

No Windows, foi usada uma ponte para a chamada semântica do njsscan. Ela mantém as regras e os exportadores oficiais. O container e o CI usam Linux sem essa ponte. Os tempos registrados são locais.

Não havia Docker ou WSL instalado no ambiente de preparação. O Compose foi preparado, mas seu teste do zero precisa ser realizado antes da aula. Datree e StackHawk não foram executados. Os prints do Anexo B foram tirados das saídas reais dos scanners e do gate local; eles não comprovam execuções no GitHub Actions.

Os integrantes precisam fazer a revisão conjunta, executar o laboratório e ensaiar. Esta preparação não afirma que Aquiles já revisou, que os dois já apresentaram ou que houve contribuições de ambos no histórico Git. Essas ações precisam ocorrer de verdade.

Dados medidos

Teste	Inseguro (s)	Corrigido (s)
njsscan-json	7.494	7.177
njsscan-sarif	7.558	7.307
npm-audit	0.894	0.828

Anexo B. Evidências locais

As capturas abaixo mostram o relatório oficial do njsscan, a saída textual do npm audit e o resultado JSON do gate. São saídas reais da execução local. Caminhos de pastas ficaram fora do enquadramento.

Figura 1 - njsscan: XSS na versão insegura

RULE ID	express_xss	
OWASP-WEB	A1: Injection	
CWE	CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')	
DESCRIPTION	Untrusted User Input in Response will result in Reflected Cross Site Scripting Vulnerability.	
SEVERITY	ERROR	
FILES	File	app.js
	Match Position	3 - 43
	Line Number(s)	8: 9
	Match String	const nome = req.query.nome; res.send(' Olá, ' + nome + ' ');

Anexo B. Evidências locais (continuação)

Figura 2 - njsscan: execução de comando

RULE ID	generic_os_command_exec	
OWASP-WEB	A1: Injection	
CWE	CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')	
DESCRIPTION	User controlled data in 'child_process.exec()' can result in Remote OS Command Execution.	
SEVERITY	ERROR	
FILES	File	app.js
	Match Position	3 - 5
	Line Number(s)	12: 14
	Match String	exec(req.query.cmd, function (erro, saida) { res.type('text').send(erro ? 'Erro' : saida); });

Anexo B. Evidências locais (continuação)

Figura 3 - njsscan: uso de eval

RULE ID	eval_nodejs	
OWASP-WEB	A1: Injection	
CWE	CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')	
DESCRIPTION	User controlled data in eval() or similar functions may result in Server Side Injection or Remote Code Injection	
SEVERITY	ERROR	
FILES	File	app.js
	Match Position	21 - 41
	Line Number(s)	17
	Match String	const resultado = eval(req.query.expr);

Anexo B. Evidências locais (continuação)

Figura 4 - npm audit: versão insegura

npm audit - versão insegura

Saída real da execução local · `npm audit --audit-level=high --ignore-scripts`

```
# npm audit report

lodash <=4.17.23
Severity: high
Command Injection in lodash - https://github.com/advisories/GHSA-35jh-r3h4-6jhm
Prototype Pollution in lodash - https://github.com/advisories/GHSA-p6mc-m468-83gw
Regular Expression Denial of Service (ReDoS) in lodash - https://github.com/advisories/GHSA-29mw-wpgm-hmr9
lodash vulnerable to Code Injection via `_.template` imports key names -
https://github.com/advisories/GHSA-r5fr-rjxr-66jc
lodash vulnerable to Prototype Pollution via array path bypass in `_.unset` and `_.omit` -
https://github.com/advisories/GHSA-f23m-r3pf-42rh
lodash has Prototype Pollution Vulnerability in `_.unset` and `_.omit` functions -
https://github.com/advisories/GHSA-xxjr-mmjv-4gpg
fix available via `npm audit fix --force`
Will install lodash@4.18.1, which is outside the stated dependency range

1 high severity vulnerability

To address all issues, run:
  npm audit fix --force
```

Anexo B. Evidências locais (continuação)

Figura 5 - Gate bloqueado

Gate - versão insegura

Saída real da execução local - execução do gate na versão insegura

```
{
  "alvo": "inseguro",
  "status": "BLOQUEADO",
  "exit_code": 1,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 3,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 1,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  },
  "achados_sast": [
    {
      "regra": "eval_nodejs",
      "nivel_original": "ERROR",
      "nivel_gate": "HIGH",
      "ocorrencias": 1,
      "cwe": "CWE-95: Improper Neutralization of Directives in Dynamically Evaluated Code ('Eval Injection')",
    },
    {
      "regra": "express_xss",
      "nivel_original": "ERROR",
      "nivel_gate": "HIGH",
      "ocorrencias": 1,
      "cwe": "CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')",
    },
    {
      "regra": "generic_os_command_exec",
      "nivel_original": "ERROR",
      "nivel_gate": "HIGH",
      "ocorrencias": 1,
      "cwe": "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')",
    }
  ]
}
```

Anexo B. Evidências locais (continuação)

Figura 6 - njsscan: versão corrigida

njsscan - versão corrigida

Saída real da execução local · njsscan app.js --json

```
{
  "errors": [],
  "njsscan_version": "1.0.0",
  "nodejs": {},
  "templates": {}
}
```

Anexo B. Evidências locais (continuação)

Figura 7 - npm audit: versão corrigida

npm audit - versão corrigida

Saída real da execução local · npm audit --audit-level=high --ignore-scripts

```
found 0 vulnerabilities
```

Anexo B. Evidências locais (continuação)

Figura 8 - Gate aprovado

Gate - versão corrigida

Saída real da execução local - execução do gate na versão corrigida

```
{
  "alvo": "corrigido",
  "status": "APROVADO",
  "exit_code": 0,
  "politica": "SAST ERROR = HIGH por decisão do grupo; SCA HIGH/CRITICAL bloqueiam",
  "contagens": {
    "sast_high": 0,
    "sast_moderate": 0,
    "sast_low": 0,
    "sca_high": 0,
    "sca_critical": 0,
    "sca_moderate": 0,
    "sca_low": 0
  },
  "achados_sast": []
}
```